

iOSDC Japan 2026 ・ Regular Talk ・ 40 min

映像変換サーバーなしで iPhone端末内で HLSを生成してライブ配信

Hikaru Sato @SatoHikaruDev

playlist.m3u8

init.mp4

000001.m4s

01

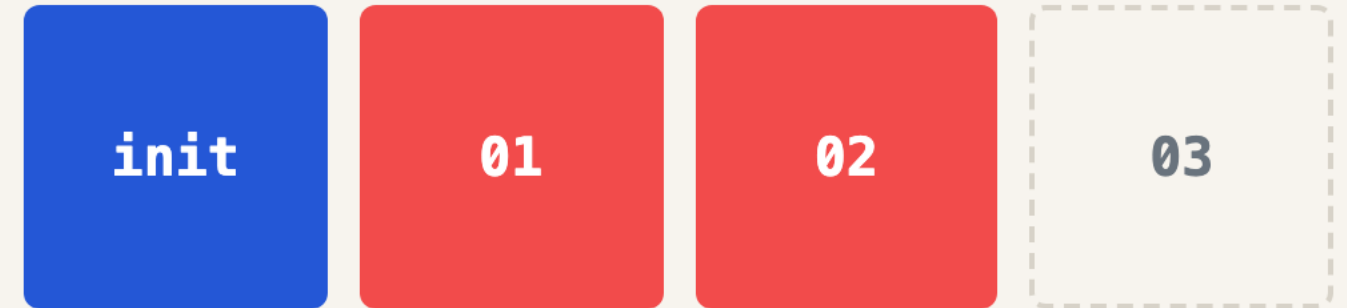
まず、HLSで 「ライブ」になる仕組み

短い動画ファイル + 更新され続けるplaylist

— HLS LIVE IN ONE SENTENCE

HLSは、短い動画と 更新されるplaylistをHTTPで配る

```
#EXTM3U
#EXT-X-MAP: init.mp4
#EXTINF: 2.000
seg/000001.m4s
#EXTINF: 2.000
seg/000002.m4s
```



— THREE FILE TYPES

HLSライブ配信を構成する、3種類のファイル

M3U8	MP4	M4S
playlist.m3u8 更新される再生順	init.mp4 codec / track情報	000001.m4s 2秒前後の映像と音声

playlistが、初期化情報とメディア断片を結びつける

— WHY IT IS LIVE

playlistが増えるたび、 Playerは次のsegmentを取得する



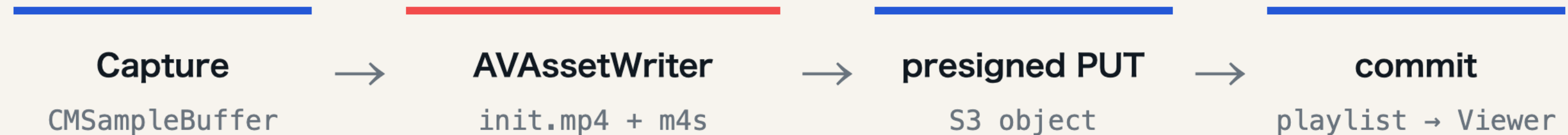
— FROM HLS TO THIS TALK

今回は、segmentを作る場所をiPhoneへ移す



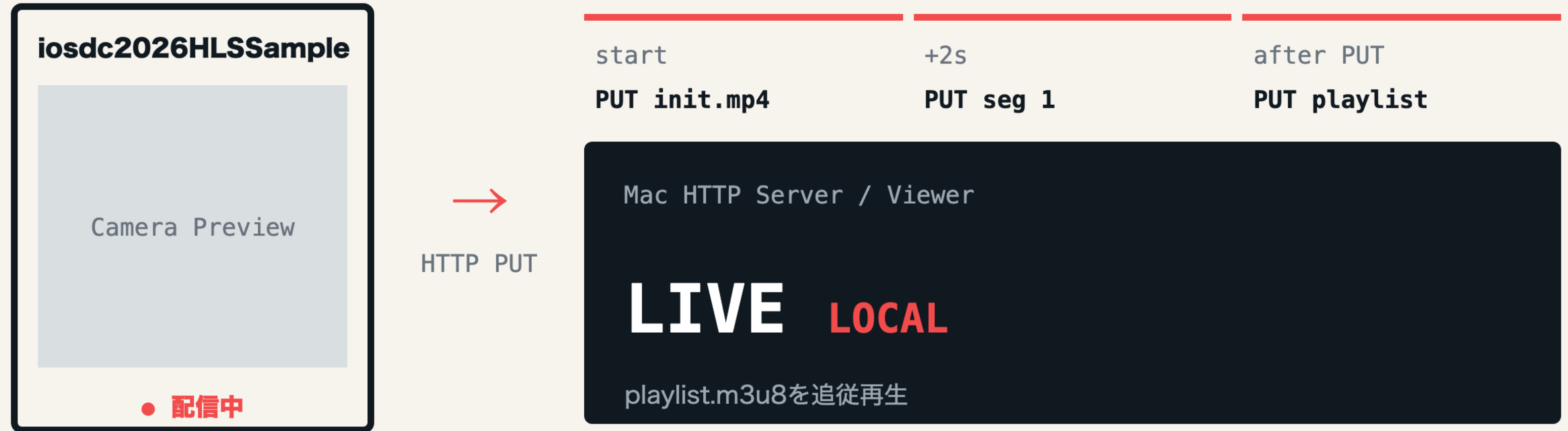
— THIS TALK'S IMPLEMENTATION

iPhoneが生成したHLSを 撮影中からS3へ公開する



— LIVE DEMO · PUBLIC SAMPLE

仕組みが見えたところで、最小構成を動かす



Macは「PUTされたファイルを残すだけ」

Mac

```
$ python3 server/server.py
```

Viewer: <http://localhost:8080>

iPhone server URL: <http://192.168.x.x:8080>

PUT [init.mp4 / m4s / m3u8](#)

DISK [server/data/streams/...](#)

GET 同じファイルをViewerへ返す

「配信開始」で、iPhoneがHLS生成を始める

● 配信開始



Camera + Mic

CMSSampleBuffer



AVAssetWriter

fMP4 Data

elapsed 0.0 → segment 0 → pending upload 0 →

最初に届くのは、1回だけのinit.mp4

t ≈ 0

initialization callback



init.mp4

codec / track / timescale



PUT /streams/{id}/init.mp4

media segmentより先に、再生の前提を保存する

以後は約2秒ごとに、2つのPUTが続く

segment 1

PUT 000001.m4s

THEN

PUT playlist.m3u8

segment 2

PUT 000002.m4s

THEN

PUT playlist.m3u8

segment N

PUT 00000N.m4s

THEN

PUT playlist.m3u8

playlistが指すのは、保存に成功したsegmentだけ

Viewerは最新streamを見つけ、playlistを追いかける

1秒ごと

GET /api/streams



最新を選択

playlistURL



LIVE

Safari / hls.js

STREAM stream-... SEGMENTS 1 → 2 → 3 PLAYLIST /streams/.../playlist.m3u8

デモ後に残る、3種類のHLSファイル

```
server/data/streams/stream-20260818-225315-C7A2AB11/
```

└─ init.mp4	1,136 bytes
└─ playlist.m3u8	682 bytes
└─ seg/	18 files
000001.m4s 000002.m4s ... 000018.m4s	

サーバーに残るのは、変換前の映像ではなく「再生可能なHLSオブジェクト」

撮影後に待つほど、 共有したい瞬間から遠ざかる

子どもの動画を家族へ送る。

撮影中から届き始めれば、終了後の待ち時間を減らせる。

撮影終了後

recording.mov

大きな1ファイルを送信



撮影中

init.mp4

001.m4s

002.m4s

小さく区切って順次送信

— BIG PICTURE · FOUR BOUNDARIES

今回の実装は、4つの境界を順番に越える

01 Capture

camera / mic → sample



02 Package

sample → fMP4



03 Publish

Data → object



04 Play

playlist → Viewer

今回の難所は、各境界で「順序」を失わないこと

— ROUTE

sample bufferをS3公開まで追う

01

HLSの最小形

02

iOSの責務分割

03

Captureと時刻

04

fMP4 segment

05

**S3 uploadと
commit**

Public at iOSDC 2026 / iosdc2026HLSSample

— SAME SHAPE, DIFFERENT DESTINATION

サンプルと本番は、同じHLSを別の場所へ置く

Sample	iPhone	HTTP PUT	Mac filesystem	HTTP GET	Local Viewer
--------	--------	----------	----------------	----------	--------------

MomentNow	iPhone	presigned PUT	Amazon S3	CloudFront	Web Viewer
-----------	--------	---------------	-----------	------------	------------

init.mp4とm4sのバイト列は、どちらもiPhoneが生成する

— CONTROL PLANE

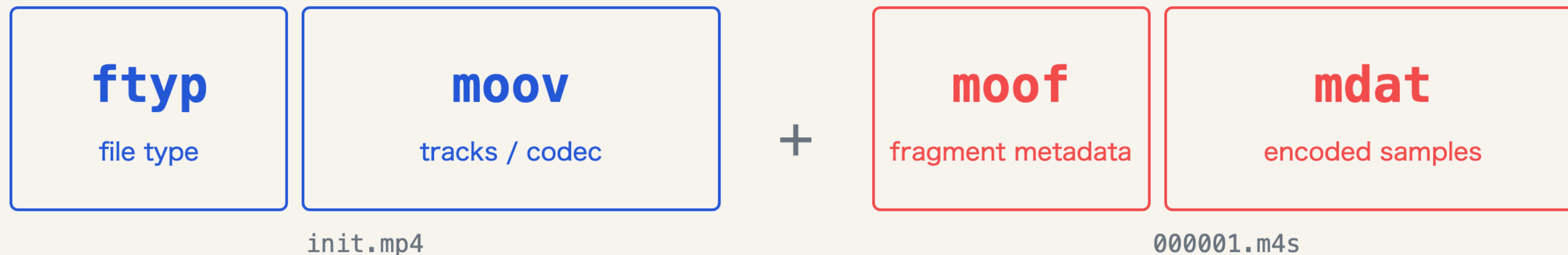
公開タイミングは、playlistが制御する



「保存できた」と「再生リストに載せた」を分ける

— FRAGMENTED MP4

fMP4は「設定」と「再生可能な断片」を分ける



— RESPONSIBILITY

AVAssetWriterはsegmentを作る。playlistは作らない。

AVFoundation

encode + segment

- H.264 / AAC
- init.mp4
- separable m4s

App

seq + PUT + commit

- sequence番号
- S3 upload
- 公開可能なseqを通知

02

この仕組みを iOSのコードへ分解する

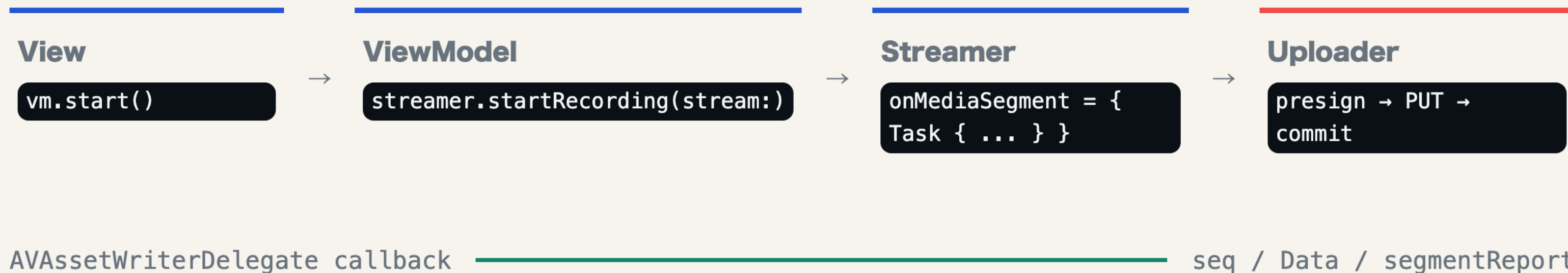
From a working stream to the AVFoundation pipeline

— FOUR OWNERS

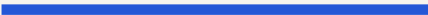
本番は4つの型で、生成と公開を分ける

UI	LiveStreamViewModel	ready / streaming / completed
ORCHESTRATE	LiveHLSStreamer	recorder callbackをuploadへ接続
MEDIA	LiveSegmentRecorder	capture / writer / media clock
UPLOAD	HLSUploadCoordinator	presign / PUT / commit

配信開始タップで、生成とuploadをつなぐ



Queueとactorは、守る状態が違う

MainActor	ViewModelの表示状態	
sessionQueue	AVCaptureSessionの構成・start / stop	
writingQueue	sample順序・AVAssetWriter状態	
HLSUploadCoordinator actor	upload数・commit状態・終了状態	

1本の巨大なロックではなく、責任ごとに直列化する

— START ORDER

callbackを接続してから、Writerを開始する

```
1 let channel = HLSUploadEventChannel()
2
3 recorder.onInitSegment = { [channel] data in
4     channel.yield(.initialization(data))
5 }
6 recorder.onMediaSegment = { [channel, fragmentSeconds] seq, data, _ in
7     channel.yield(.media(seq: seq, data: data, durationSec: fragmentSeconds))
8 }
9
10 uploadTask = Task {
11     await coordinator.consume(channel.stream, channel: channel)
12 }
13 try await recorder.startRecording()
```

最初のinitialization callbackを取りこぼさない

03

Captureが メディアの時計を作る

Camera, microphone, sample buffers, and one shared timeline

— CAPTURE TOPOLOGY

AVCaptureSessionが、2つのdeviceを2つのDataOutputへつなぐ

Back camera

AVCaptureDeviceInput

Microphone

AVCaptureDeviceInput

AVCaptureSession

VideoDataOutput

CMSampleBuffer

AudioDataOutput

CMSampleBuffer

HLSegmentRecorder.setupCaptureSessionLocked()

— WHY DATA OUTPUT

完成ファイルではなく、sample bufferを使う

MovieFileOutput

録画ファイルを作る

高レベルで便利



Video / Audio DataOutput

各sampleを受け取る

Writerへ逐次append

video CMSampleBuffer

audio CMSampleBuffer

same writingQueue

— INPUT FORMAT

CameraのNV12をWriterがH.264に圧縮

Capture

420f / NV12

pixel buffer

append

Writer

H.264 High

fMP4 sample

```
1 videoOutput.videoSettings = [  
2     kCVPixelBufferPixelFormatTypeKey as String:  
3     kCVPixelFormatType_420YpCbCr8BiPlanarFullRange  
4 ]
```

HLSegmentRecorder.setupCaptureSessionLocked()

Writerへ届く前に、2つのguardで落とす

captureOutput(_:didOutput:from:)

isWriting == true

録画中だけ

CMSampleBufferDataIsReady

利用可能なsampleだけ

startWriterIfNeeded
→ append

```
1 guard isWriting else { return }
2 guard CMSampleBufferDataIsReady(sampleBuffer) else { return }
```

— SESSION START

最初のvideo sampleで、全trackを開始

audio sample

まだappendしない

first video sample

startWriting

startSession(at: 10s)

video + audio共通

```
1 guard output === videoOutput else { return }
2 guard !didStartSession, let writer else { return }
```

— TWO CLOCKS

Capture PTSはHLSの開始時刻ではない

Capture PTS

58342.31

|

|

...

– firstVideoPTS + 10s

HLS timeline

10.00

|

|

...

— ONE DELTA

時刻補正は、全sampleへの平行移動

delta = 10s - firstVideoPTS

adjustedPTS = sourcePTS + delta

VideoとAudioを別々に補正しない

video	58342.31	same delta	video	10.00
audio	58342.29		audio	9.98

相対差を保ったまま、Writerのtime rangeへ移す

— APPEND READINESS

詰まったsampleは、待たずに落とす

```
1 if output === videoOutput {  
2     guard videoInput.isReadyForMoreMediaData else { return }  
3     didAppend = videoInput.append(adjustedSampleBuffer)  
4 } else {  
5     guard audioInput.isReadyForMoreMediaData else { return }  
6     didAppend = audioInput.append(adjustedSampleBuffer)  
7 }
```

push source

×

real-time writer

→

bounded latency

04

AVAssetWriterが sampleをfMP4へ分ける

Four HLS settings and a segment delegate

— URL-LESS WRITER

URLなしで、Dataをdelegateから受け取る

```
1 let writer = AVAssetWriter(contentType: .mpeg4Movie)
2 writer.shouldOptimizeForNetworkUse = true
3 writer.delegate = self
```

AVAssetWriter

MP4 content type

initialization Data

separable Data

— FOUR HLS KNOBS

HLS出力は、4つのpropertyで有効になる

- | | | |
|----|---|-----------------|
| 01 | <code>outputFileTypeProfile</code> | Apple HLS向けfMP4 |
| 02 | <code>preferredOutputSegmentInterval</code> | 希望するsegment間隔 |
| 03 | <code>initialSegmentStartTime</code> | 最初のsegment開始時刻 |
| 04 | <code>delegate</code> | 生成されたDataの受け口 |

`LiveSegmentRecorder.setupWriters_locked()`

— PREFERRED INTERVAL

2秒は「希望」。境界は前後する

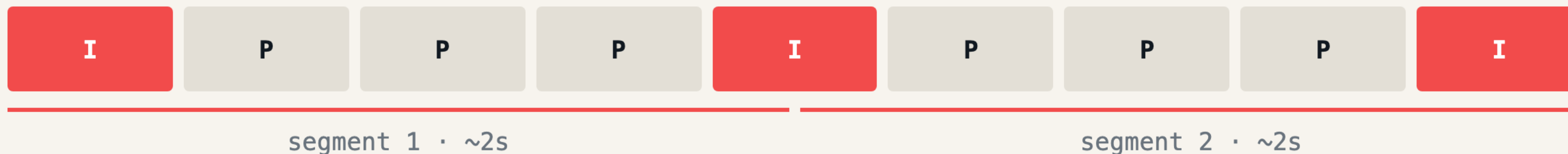


```
1 writer.preferredOutputSegmentInterval = CMTime(  
2     seconds: config.segmentSeconds,  
3     preferredTimescale: 600  
4 )
```

現在は2.0秒固定。segmentReport反映は改善項目

— KEYFRAME ALIGNMENT

2秒で切るなら、2秒以内にIDRを用意する

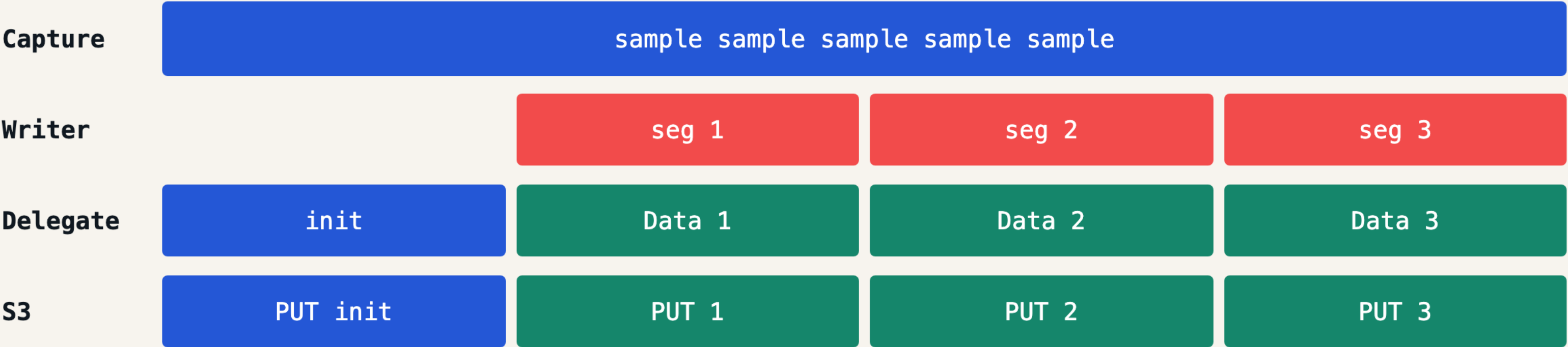


1 `AVVideoMaxKeyFrameIntervalDurationKey: config.segmentSeconds`

segmentTypeでinitとmediaを分ける

```
1 switch segmentType {
2   case .initialization:
3     onInitSegment?(segmentData)
4   case .separable:
5     segmentIndex += 1
6     onMediaSegment?(segmentIndex, segmentData, segmentReport)
7   @unknown default:
8     break
9 }
```

Captureを止めず、約2秒ごとにDataを出す



Writerのfinishを待たず、生成とS3 uploadを並行できる

— BACKPRESSURE DECISION

Writerが詰まっても、Captureを待たせない

DataOutput

real-time push

`isReadyForMoreMediaData`

true
append

false
return /
drop

長所: latencyとmemoryがbounded

代償: frame dropを観測していない

再生開始は、init PUTと最初のcommitの後

start	network	~2s	after PUT
init Data	PUT init	PUT seg 1	commit seq 1

init object exists

playlist contains seq 1

seg 1 object exists

Viewer can start

— COPY, UPLOAD, COMMIT

playlist状態は、PUT成功後にだけ進める

```
1 var nextManifest = manifest
2 nextManifest.addSegment(seq: seq, durationSec: durationSec)
3
4 try await client.putPlaylist(streamId: streamId, text: nextManifest.text)
5 manifest = nextManifest
```

current
segments 1...N

copy

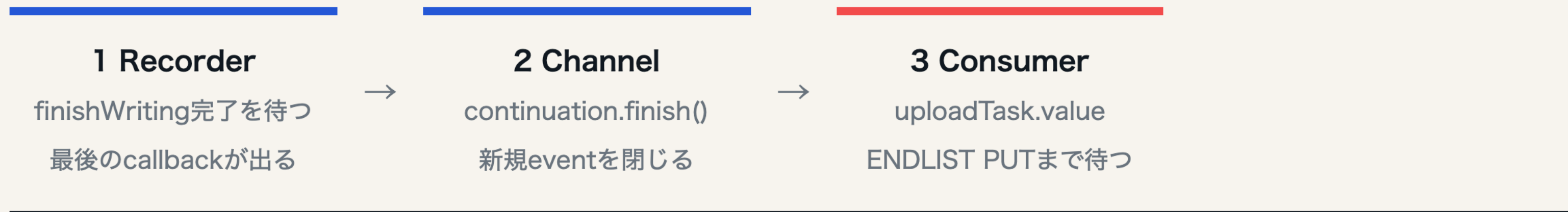
candidate
+ segment N+1

PUT
2xx

commit
current = candidate

— DRAIN BEFORE FINISH

stopは、3つの非同期処理を順番に閉じる



「撮影停止」と「配信終了」は、同じ瞬間ではない

05

端末で生成したDataを S3へ順番に公開する

Presign, PUT, commit, and finish from the iOS point of view

— PREPARE BEFORE MEDIA

segment前にstreamIdと再生URLを準備

createStream

streamId / prefix

→

ticket

playbackUrl

→

permission

camera / microphone

→

ready

previewSession

```
1 if ticket == nil {  
2     ticket = try await apiClient.ticket(streamId: stream.streamId)  
3 }  
4 await requestPermissionsAndSetup()  
5 try await streamer?.start()
```

— INITIALIZATION SEGMENT

init.mp4は、最初に1度だけPUT

```
1 guard !didUploadInit else { return }
2 let ticket = try await api.presign(
3     streamId: stream.streamId, kind: "init", seq: nil
4 )
5 try await api.put(
6     to: URL(string: ticket.putUrl)!, data: initData,
7     contentType: "video/mp4"
8 )
9 didUploadInit = true
```

— MEDIA SEGMENT

media  とは presign → PUT → commit

```
1 let ticket = try await api.presign(  
2     streamId: stream.streamId, kind: HLSUploadKind.segment,  
3     seq: seq  
4 )  
5 try await api.put(  
6     to: URL(string: ticket.putUrl)!, data: segmentData,  
7     contentType: HLSContentType.segmentM4S  
8 )  
9 _ = try await api.commit(  
10     streamId: stream.streamId, seq: seq, durationSec: dur,  
11     isLast: false, targetDurationSec: targetDurationSec  
12 )
```

— NO AWS CREDENTIAL ON DEVICE

iPhoneへ渡すのは、短期のPUT URLだけ

DO NOT

AWS credentialを内包

漏えい範囲が広い



PRESIGNED URL

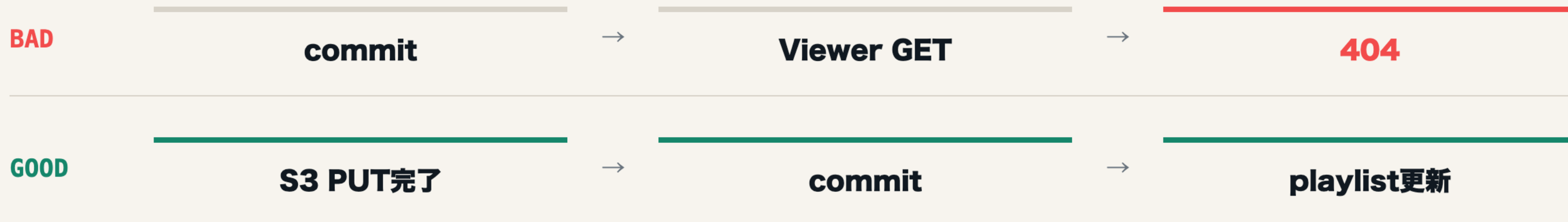
object単位のPUT権限

streamId / kind / seqで制約

```
1 var request = URLRequest(url: presignedURL)
2 request.httpMethod = "PUT"
3 request.setValue(contentType, forHTTPHeaderField: "Content-Type")
4 request.httpBody = data
```

— PUBLISH AFTER BYTES

PUT成功より先に、playlistへseqを出さない



存在するsegmentだけを、再生可能として公開する

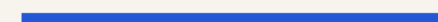
— ACTOR STATE

Actorは、uploadと終了の状態をまとめて守る

didUploadInit init.mp4の重複PUTを防ぐ



lastCommittedSeq 公開済み最大seq



inUploadSegmentCount 停止時に待つbacklog



isFinishing 遅れて完了したsegmentでもENDLISTを再通知



actor内のawaitは再入可能。完了順ではなくseqを正とする

— STOP CONTRACT

stop後は pending uploadが0になるまで完了にしない



```
1 while await streamer.hasPendingUploads() {  
2     try? await Task.sleep(for: .milliseconds(500))  
3 }
```

06

iOSの非同期境界を ライブ配信として閉じる

Callback, task, upload backlog, and publication order

— CALLBACK TO TASK

callbackでは待たず、TaskへDataを引き渡す

LiveSegmentRecorder

init Data / media Data

WRITING QUEUE

callbackを返す

次のsample処理へ

SWIFT TASK

Uploaderをawait

presign / PUT / commit

initialization → uploadInitIfNeeded

separable → uploadSegment

segmentReport → duration改善余地

— THREE REQUESTS

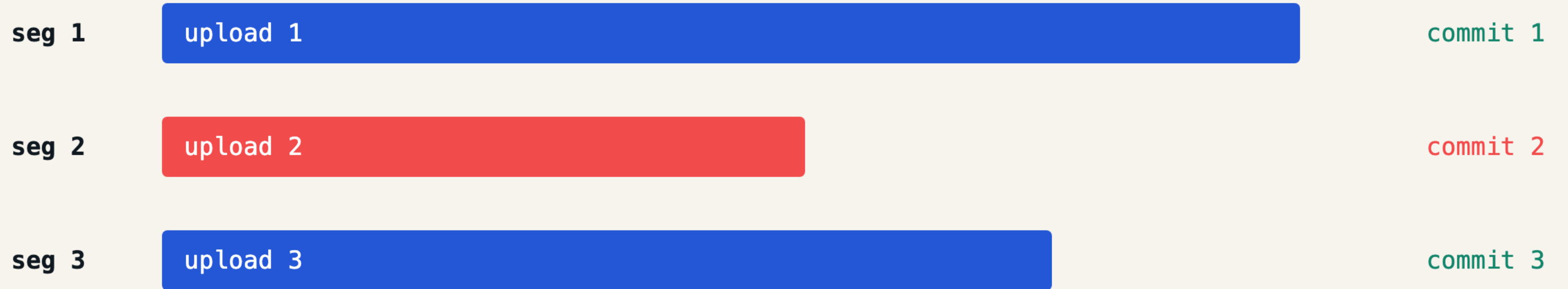
presign ・ PUT ・ commitは、役割が違う

REQUEST	DESTINATION	ROLE
POST /presign	API	seq用PUT URLを発行
PUT putUrl	S3	video/mp4 Dataを保存
POST /commit	API	seqをplaylistへ公開
GET playbackUrl	CloudFront	Viewerがplaylistを取得

iOSはplaylist本文を書かず、「公開してよいseq」をcommitする

— OUT-OF-ORDER RACE

uploadの完了順は、生成順とは限らない



completion **2, 1, 3** $\neq$ playback **1, 2, 3**

commit APIは、欠番より先をplaylistへ出さない契約が必要

— SAMPLE → PRODUCTION

本番化では、4つの運用機能を足す

AUTH

user / group ownership

PRESIGN

object単位の短期PUT権限

COMMIT

ETag / If-Matchでplaylist競
合制御

DELIVERY

CloudFront / ticket /
status

AVAssetWriterの後ろを差し替えれば、同じiOS pipelineを使える

— BACK TO THE DEMO

冒頭のデモで増えていたのは
「ファイル」と「再生可能な順序」

init.mp4 + 000001.m4s + playlist更新 = LIVE

— FIT, NOT UNIVERSAL

この構成が効くのは、用途を絞ったとき

GOOD FIT

少人数・短時間・単一品質

新しいiPhone / 安定した上り回線

USE A MEDIA PLATFORM

大規模・長時間・ABR・厳しいSLA

発熱、電池、回線変動を吸収したい

「端末でできる」と「端末でやるべき」は別の判断

— TAKEAWAYS

成立条件は、4つの順序をそろえること

01 Clock

Capture PTSをWriterのtimelineへ移す

02 Boundary

IDRとsegment intervalをそろえる

03 Upload

DataをS3へPUTしてからcommitする

04 Finish

pendingが0になるまで完了にしない

ありがとうございました

zenn.dev/hs7/articles/080eac650f65ba



APPENDIX

時間が余ったとき／Q&A用

設定値・時刻補正・サーバー運用の詳細

9ファイルで、生成から再生まで追える

HLSSegmentRecorder.swift	capture / encode / segment	SampleHLSStreamer.swift	recorderとuploadを接続
HLSUploadCoordinator.swift	順序 / retry / finish	HLSManifest.swift	playlist state
HTTPHLSClient.swift	PUT / health check	SampleStreamViewModel.swift	permission / UI state
ContentView.swift	操作と観測	server.py	atomic replace / GET
static/app.js	latest stream / hls.js		

— CONFIG

上り回線に合わせて、配信品質を3段階から選ぶ

```
1 let profile: (CGSize, Int) = switch quality {  
2 case .high:    (.init(width: 720, height: 1280), 2_500_000)  
3 case .medium:  (.init(width: 720, height: 1280), 1_500_000)  
4 case .low:     (.init(width: 480, height: 854),    900_000)  
5 }
```

NWPath

constrained / expensive

presign

API latency

probe PUT

upload throughput

2.0 sec

segment target

カメラ構成前に、録音用AudioSessionを有効化

```
1 let audio = AVAudioSession.sharedInstance()
2 try audio.setCategory(
3     .playAndRecord,
4     mode: .videoRecording,
5     options: [.defaultToSpeaker, .allowBluetoothHFP]
6 )
7 try audio.setActive(true)
```

Capture

camera + microphone

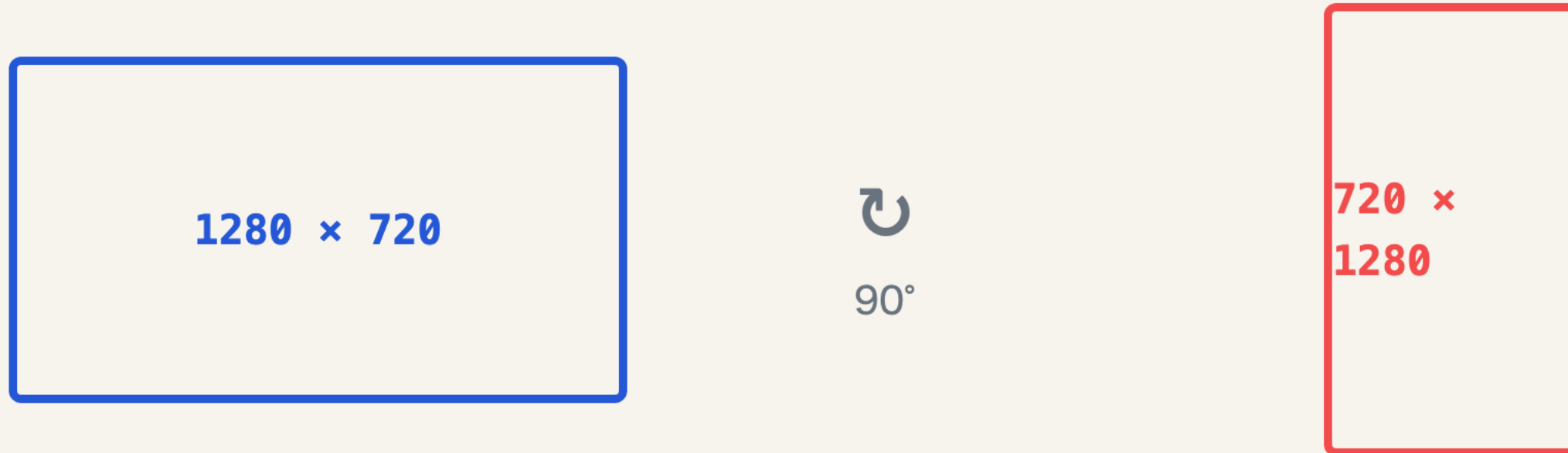
Route

speaker + Bluetooth HFP

AVCaptureSessionだけでなく、AVAudioSessionのrouteも配信品質に影響する

— ORIENTATION

縦向きは、video connectionへ90度を指定



```
1 if let connection = videoOutput.connection(with: .video),
2   connection.isVideoRotationAngleSupported(90) {
3   connection.videoRotationAngle = 90
4 }
```

— COPY TIMING

Media bytesは変えず、timing metadataをコピーする

```
1 let timingInfos = try sampleTimingInfos().map { info in
2     var adjusted = info
3     adjusted.presentationTimeStamp =
4         info.presentationTimeStamp + offset
5     if info.decodeTimeStamp.isValid {
6         adjusted.decodeTimeStamp = info.decodeTimeStamp + offset
7     }
8     return adjusted
9 }
10 let copied = try CMSampleBuffer(copying: self, withNewTiming: timingInfos)
```

PTSと、有効なDTSを同じ量だけ動かす

— STOP IN THE SAME CLOCK

開始と終了を、同じ補正後timelineで閉じる

lastAdjustedPTS

最後にappendした時刻

→

endSession

media rangeを閉じる

→

markAsFinished

video / audio

→

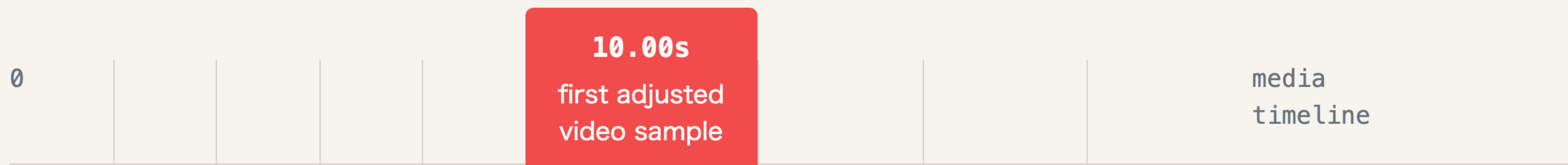
finishWriting

最後のsegmentをflush

```
1 if lastAdjustedPTS.isValid {  
2     writer.endSession(atSourceTime: lastAdjustedPTS)  
3 }
```

— INITIAL START TIME

Writerの最初のsegmentは、10秒起点に固定



```
1 private let startTimeOffset = CMTime(value: 10, timescale: 1)
2 writer.initialSegmentStartTime = startTimeOffset
```

配信用videoは、互換性と上り帯域を優先する

KEY	VALUE	INTENT
AVVideoCodecKey	H.264	広い再生互換性
Width / Height	480p / 720p	quality別に選択
AverageBitRate	0.9–2.5 Mbps	上り回線へ追従
ProfileLevel	High Auto	encoder profile
FrameReordering	false	decode順を単純化

AudioはAAC mono、品質別に48-96kbps

FORMAT	CHANNEL	SAMPLE RATE	BITRATE
AAC	mono	44.1 kHz	48-96 kbps

```
1  [  
2    AVFormatIDKey: kAudioFormatMPEG4AAC,  
3    AVNumberOfChannelsKey: 1,  
4    AVSampleRateKey: 44_100,  
5    AVEncoderBitRateKey: config.audioBitrate  
6  ]
```

— DURATION CAVEAT

現在は2秒固定。実測時間は次の改善点

CURRENT

durationSec = 2.0

fragmentSecondsをcommit

NEXT

segmentReportから実測

EXTINFへ正確な長さ

```
1 recorder.onMediaSegment = { [uploader, fragmentSeconds] seq, data, _ in
2     try? await uploader.uploadSegment(
3         stream: stream, seq: seq, segmentData: data,
4         durationSec: fragmentSeconds
5     )
6 }
```

— RETRY POLICY

失敗時は、同じobjectを最大3回まで再送する



segment PUTが失敗したら、そのsegmentをplaylistへ載せない

— UI OBSERVATION LOOP

ViewModelは300msごとに、配信状態をsnapshotする

```
1 monitorTask = Task {  
2     while !Task.isCancelled {  
3         elapsedSeconds = streamer.recordedSeconds  
4         if let snapshot = await streamer.currentSnapshot() {  
5             apply(snapshot)  
6         }  
7         try? await Task.sleep(for: .milliseconds(300))  
8     }  
9 }
```

elapsed

segmentCount

pendingUploadCount

playlistText

error

PUT中のファイルを、Viewerへ見せない



playlistの途中状態や、半分だけのm4sをGETさせない

cache方針は、playlistとsegmentで分ける

OBJECT	CACHE	HTTP
playlist.m3u8	no-store / no-cache	毎回最新を取得
init.mp4 / m4s	max-age=31536000	Range / 206対応
static assets	max-age=86400	Viewer UI

この区別は、S3 + CloudFrontでもそのまま使う

iPhone側で、upload backlogと速度を測る

METRIC	MEANING	USE
pendingSegmentCount	未完了upload数	stop待機 / backlog
averageRoundTripSec	presign + PUT	segment間隔と比較
averageUploadDurationSec	PUT所要時間	回線劣化を検知
averageUploadThroughputBps	送信bitrate	品質選択の根拠

平均は smoothingFactor = 0.25 の指数移動平均